Lundi 16/11, après-midi — Fonctions en paramètre (callback) et map

Kevin Bardoux

lundi 16 novembre 2026

Table des matières

Objectifs 2
Après-midi — Callback et map 3
Votre dossier de la semaine (5 min) 3
Rappel — Les fonctions (10 min) 3
Rappel — vérification 4
Callback et map (15 min) 4
Callback et map, en image 5
map, case par case 5
Callback et map, l'exemple minimal 5
Callback et map — attention et vérification 6
Pas à pas ensemble — Les noms des musées (40 min) 7
Variante à faire seuls 10
Enregistrer et envoyer (2 min) 10
Exercices — Callback et map 10
Palier 1 — Doubler, puis les longueurs (30 min) 11
Palier 2 — Majuscules et prix TTC (40 min) 11
Palier 3 — map à la main (30 min) 12
Enregistrer et envoyer (2 min) 13
À retenir 14
Si le temps le permet 14
Débrief 14
Sources 14

> **Note**
>
> **Compétences visées** : C4 · résolution de problème **Livrable** : le pas à pas `pas-a-pas-map` envoyé sur GitHub, et `node --test l1-doubler.test.js` au vert.

> **! Le minimum du jour**
>
> - Le pas à pas « Les noms des musées », fait avec le groupe.
> - Le palier 1 : `node --test l1-doubler.test.js` et `node --test l2-longueurs.test.js` au vert.
>
> Les paliers 2 et 3, puis la rubrique « Si le temps le permet », sont la suite naturelle, pour ceux qui avancent.

Objectifs

- Passer une fonction en paramètre d'une autre, sans l'appeler (*callback*).
- Écrire une fonction fléchée courte.
- Transformer chaque élément d'un tableau avec `map`, sans modifier le tableau reçu.

Comment se passe l'après-midi. Une seule notion : la fonction passée en paramètre, et `map`, la méthode qui s'en sert. D'abord un rappel de 10 minutes sur les fonctions ; puis une démonstration lente ; puis nous codons ensemble, une étape à la fois ; enfin, vos exercices, en trois paliers.

Cette semaine, trois paliers d'exercices. Après chaque live coding, les exercices sont rangés en trois paliers :

- **Palier 1** : le minimum, guidé. Tout le monde le fait.
- **Palier 2** : recommandé, semi-guidé. Vous avez les étapes principales.
- **Palier 3** : autonome. Une consigne ouverte : il faut chercher dans la documentation.

Les tests des paliers 1 et 2 sont dans `nom.test.js` ; ceux du palier 3, dans `nom.defi.test.js`. Si vous vous arrêtez au palier 2, aucun test ne reste rouge.

Après-midi — Callback et map

Votre dossier de la semaine (5 min)

1. Téléchargez l'archive de la semaine.
2. En ligne de commande : créez `semaine-07` dans `~/dwwm/dwwm-tp`.
3. Extrayez l'archive dans `semaine-07`, puis ouvrez-y VS Code.
4. Dans le terminal de VS Code : `npm install`.
5. **Vous devez voir**, à la fin : `found 0 vulnerabilities`.

L'archive n'a pas de dossier à l'intérieur : ses fichiers vont directement dans `semaine-07`. Sous Windows, *Extraire tout…* propose un dossier `s07-exercices` : remplacez ce chemin par celui de `semaine-07`. **Réussi si** : `semaine-07` contient `package.json`, `l1-doubler.js` et le dossier `pas-a-pas-map`.

Les commandes du terminal (créer un dossier, aller dedans, ouvrir VS Code avec `code .`) sont dans la fiche aide-mémoire Linux du Drive partagé de la formation et dans la fiche de révision de la semaine 4. `code .` est inconnue ? Ouvrez VS Code, puis *Fichier* > *Ouvrir le dossier…*, et choisissez `semaine-07`.

La fin de `npm install` parle de vulnérabilités ? **Ne lancez pas** `npm audit fix --force` : lisez l'entrée « npm parle de vulnérabilités » de la fiche, et prévenez le formateur. Toute la semaine : VS Code ouvert sur `semaine-07`, un commit par exercice réussi, un `git push` à la fin de chaque demi-journée.

Rappel — Les fonctions (10 min)

- **Déclarer**, puis **appeler** avec des valeurs.
- `return` : la valeur que la fonction renvoie.

Ce matin, la séance PAF a pris la place du cours : nous reprenons ce qu'il faut savoir des fonctions avant d'aller plus loin. Vous les avez vues en semaine 3.

- **Déclarer** une fonction, c'est lui donner un nom, des paramètres (les valeurs qu'elle attend) et un corps (ce qu'elle fait). Rien ne s'exécute encore.

- **Appeler** une fonction, c'est écrire son nom suivi de parenthèses, avec les valeurs à lui donner : les arguments. Le corps s'exécute alors.
- `return` termine la fonction et donne sa valeur à l'endroit de l'appel. Sans `return`, la fonction renvoie `undefined` (MDN Web Docs, s. d.-c).

```
// Déclarer : un nom, un paramètre, un corps
function addTax(price) {
  // return : renvoie le prix TTC
  return price * 1.2;
}

// Appeler : 10 est l'argument, price vaut 10
console.log(addTax(10));
```

Vous devez voir, avec `node essai.js` : `12`.

Rappel — vérification

- **Vérifiez** : que renvoie `addTax(5)` ?
- Et une fonction dont on a oublié le `return` ?

Callback et map (15 min)

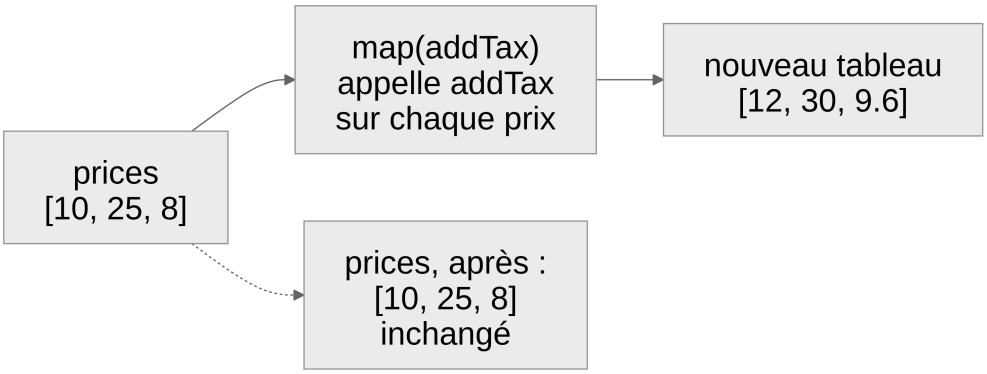
- **Pourquoi** : même parcours du tableau, action différente.
- **Callback** : une fonction donnée à une autre, qui l'appellera.
- `map` : un nouveau tableau, un résultat par élément.

Pourquoi. Mettre six noms en majuscules, calculer le prix TTC de dix prix : chaque fois, on parcourt le tableau, on applique une action à chaque élément et on range le résultat dans un nouveau tableau. Le parcours est toujours le même ; seule l'action change. On donne donc **l'action** en paramètre, sous la forme d'une fonction.

Définition. Une fonction passée en argument à une autre, pour être appelée par elle, s'appelle une fonction de rappel, ou *callback* (MDN Web Docs, s. d.-b). La méthode `map` crée un **nouveau** tableau avec les résultats de la fonction appelée sur chaque élément du tableau d'origine (MDN Web Docs, s. d.-a). Une fonction fléchée

est une écriture courte : `(paramètre) => résultat` (MDN Web Docs, s. d.-d).

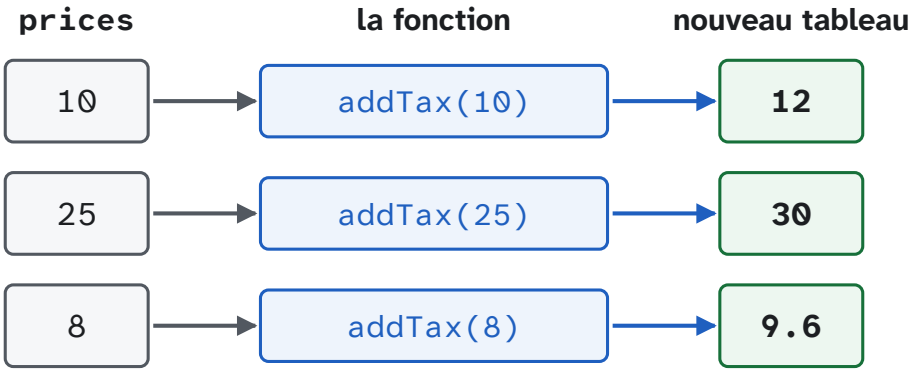
Callback et map, en image



map appelle la fonction donnée sur chaque élément, et range chaque résultat dans un nouveau tableau.

Le texte du schéma : le tableau `prices` (`10`, `25`, `8`) entre dans `map` ; `map` appelle `addTax` sur chaque prix ; un **nouveau** tableau sort (`12`, `30`, `9.6`), de même longueur ; `prices` n'a pas changé. Voyez `map` élément par élément dans le visualiseur pas à pas : avancez avec **Ligne suivante**.

map, case par case



prices après map : [10, 25, 8], inchangé

Autant de cases à la sortie qu'à l'entrée, dans le même ordre.

Figure 1. – map appelle la fonction sur chaque case et range chaque résultat dans un nouveau tableau.

Callback et map, l'exemple minimal

Le premier programme de la notion, annoté : les numéros renvoient aux explications sous le code.

```
function addTax(price) {
  return price * 1.2;
}

const prices = [10, 25, 8];
const withTax = prices.map(addTax);
const doubled = prices.map((price) => price * 2);

console.log(withTax); // [ 12, 30, 9.6 ]
```

Ligne 1

Une fonction ordinaire, comme dans le rappel : un prix entre, un prix TTC sort.

Ligne 6

`addTax` est donnée **sans parenthèses** : c'est la fonction elle-même, le *callback*. `map` l'appelle trois fois : `addTax(10)`, `addTax(25)`, `addTax(8)`.

Ligne 7

La même idée, avec une fonction fléchée écrite sur place : un paramètre, `price`, et le résultat, `price * 2`, renvoyé sans `return`.

Ligne 9

Le nouveau tableau : un résultat par élément, dans le même ordre. `prices` vaut toujours `[10, 25, 8]`.

Callback et map — attention et vérification

> ⚠ Attention
>
> `prices.map(addTax())` **appelle** `addTax` tout de suite, au lieu de la donner à `map` : `TypeError: number NaN is not a function`. Écrivez le nom seul, sans parenthèses : `prices.map(addTax)`.

- **Vérifiez** : que donne `[1, 2, 3].map((n) => n * 10)` ?
- Prédisez, puis tapez-le dans le REPL (`node`, puis `.exit`).

Pas à pas ensemble — Les noms des musées (40 min)

- Le formateur code, vous tapez en même temps, une étape à la fois.
- Vous tapez aussi les commentaires : `//` en JavaScript.
- Dossier : `pas-a-pas-map`, fichier `noms.js`.
- Perdu ? Dossier `rattrapage/map/`, une étape à la fois.

Palier 1 : guidé. Dans VS Code, ouvrez `pas-a-pas-map/noms.js`. Il contient déjà les données : six noms de musées du Grand-Est, tirés du jeu « Fréquentation des Musées de France » (Ministère de la Culture, s. d.), sous Licence Ouverte (Etalab, s. d.). Dans le terminal, allez dans le dossier : `cd pas-a-pas-map`.

Les commentaires. Dans chaque pas à pas, vous tapez le code **et** ses commentaires : ils disent ce que fait la ligne et le nom de la notion. En JavaScript, un commentaire commence par `//` ; il est sur sa propre ligne, juste au-dessus de la ligne qu'il explique. Node les ignore : ils servent à vous relire.

Avant de commencer. Lancez `node noms.js`, depuis `pas-a-pas-map`. **Vous devez voir** : les données de départ, sans erreur. Chaque étape ajoute ses lignes à la fin du fichier, et sa sortie à la fin de la console.

Étape 1 — Une fonction, un appel. À la fin du fichier, ajoutez la fonction `shout`, puis appelez-la sur le premier nom.

Avant d'exécuter : la console affichera-t-elle le nom tel quel, ou en majuscules ?

```
// Fonction : met un nom en majuscules
function shout(name) {
  // return : renvoie le nom en majuscules
  return name.toUpperCase();
}

// Appel : on donne le premier nom, on lit le résultat
console.log(shout(names[0]));
```

Enregistrez, puis lancez `node noms.js` dans le terminal, depuis `pas-a-pas-map`.

Vous devez voir, à la fin de la sortie :

```
MUSÉE UNTERLINDEN
```

Perdu ? Copiez `rattrapage/map/etape-1/noms.js` à la place de votre `noms.js`, puis repartez avec le groupe.

Étape 2 — Passer la fonction à `map`. Passez `shout` à `map`, **sans parenthèses** : c'est la fonction elle-même que vous donnez.

Avant d'exécuter : combien de noms y aura-t-il dans le nouveau tableau ?

```
// map : appelle shout sur chaque nom (callback)
const shouted = names.map(shout);
// Affiche le nouveau tableau
console.log(shouted);
```

Enregistrez, puis lancez `node noms.js` dans le terminal, depuis `pas-a-pas-map`.

Vous devez voir, à la fin de la sortie :

```
[
  'MUSÉE UNTERLINDEN',
  "ÉCOMUSÉE D'ALSACE",
  'MUSÉE ARTHUR RIMBAUD',
  "MUSÉE DE L'ARDENNE",
  'MUSÉE DU CHÂTEAU-FORT',
  'MUSÉE DE LA RENAISSANCE EN CHAMPAGNE '
]
```

Perdu ? Copiez `rattrapage/map/etape-2/noms.js` à la place de votre `noms.js`, puis repartez avec le groupe.

Étape 3 — Une fonction fléchée. Retirez les espaces autour de chaque nom, avec une fonction fléchée écrite directement dans `map`.

Avant d'exécuter : le dernier nom finit par une espace : que va-t-elle devenir ?

```
// Fonction fléchée : retire les espaces autour du nom
const trimmed = names.map((name) => name.trim());
// Affiche les noms sans espace en trop
console.log(trimmed);
```

Enregistrez, puis lancez `node noms.js` dans le terminal, depuis `pas-a-pas-map`.

Vous devez voir, à la fin de la sortie :

```
[
  'musée Unterlinden',
  "écomusée d'Alsace",
  'musée Arthur Rimbaud',
  "musée de l'Ardenne",
  'musée du château-fort',
  'musée de la Renaissance en Champagne'
]
```

Perdu ? Copiez `rattrapage/map/etape-3/noms.js` à la place de votre `noms.js`, puis repartez avec le groupe.

Étape 4 — Des longueurs, et l'original. Calculez la longueur de chaque nom nettoyé, puis affichez à nouveau `names`.

Avant d'exécuter : après trois `map`, le tableau `names` a-t-il changé ?

```
// map : la longueur de chaque nom
const lengths = trimmed.map((name) => name.length);
// Affiche les six longueurs
console.log(lengths);
// Le tableau de départ n'a pas changé
console.log(names);
```

Enregistrez, puis lancez `node noms.js` dans le terminal, depuis `pas-a-pas-map`.

Vous devez voir, à la fin de la sortie :

```
[ 17, 17, 20, 18, 21, 36 ]
[
  'musée Unterlinden',
  "écomusée d'Alsace",
  'musée Arthur Rimbaud',
  "musée de l'Ardenne",
  'musée du château-fort',
  'musée de la Renaissance en Champagne '
]
```

Perdu ? Copiez `rattrapage/map/etape-4/noms.js` à la place de votre `noms.js`, puis repartez avec le groupe.

Variante à faire seuls
Pour ceux qui ont validé l'étape avant les autres, dans le même fichier. Créez `labels` : chaque nom nettoyé, précédé de `À voir :`. Affichez-le. **Réussi si** : la console affiche six textes qui commencent par `À voir :`.

Enregistrer et envoyer (2 min)
Dans le terminal, depuis `~/dwwm/dwwm-tp/semaine-07` :

```
git status
git add pas-a-pas-map
git commit -m "Ajoute le pas à pas map"
git push
```

Vous devez voir : `git status` affiche `rien à valider, la copie de travail est propre` ; sur GitHub, le commit apparaît dans `dwwm-tp`. Le message suit la fiche aide-mémoire commit du Drive partagé (ou la fiche de révision de la semaine 4).

Exercices — Callback et map

Chaque exercice est un fichier `nom.js` qui contient sa consigne en commentaire, et un fichier de tests. Lancez toujours les tests avec le nom du fichier : `node --test l1-doubler.test.js`. N'écrivez aucun `console.log` en dehors de la fonction : les tests importent votre fichier (l'erreur « Tout le fichier de tests échoue » de la fiche

« Lire un message d'erreur »). Pour essayer votre fonction, servez-vous du REPL ou de `essai.js`.

Palier 1 — Doubler, puis les longueurs (30 min)

1. Ouvrez `l1-doubler.js` ; lisez la consigne.
2. Remplacez la ligne `return` par un `map`.
3. Lancez `node --test l1-doubler.test.js`.
4. Même chose avec `l2-longueurs.js`.

Palier 1 : guidé. Durée : 30 min (15 min par exercice).

1. Ouvrez `l1-doubler.js`. La fonction `double(numbers)` doit renvoyer un **nouveau** tableau, chaque nombre multiplié par 2.
2. Remplacez la ligne qui commence par `return` (celle du texte « À faire ») par `return numbers.map((n) => n * 2);`. Lisez-la à voix basse : « pour chaque `n`, renvoie `n * 2` ».
3. Enregistrez, puis lancez `node --test l1-doubler.test.js`.
4. Ouvrez `l2-longueurs.js`. La fonction `lengths(words)` renvoie la longueur de chaque mot. Écrivez le `map` vous-même : la longueur d'un mot `word` s'écrit `word.length`.
5. Lancez `node --test l2-longueurs.test.js`.
6. Rituel : un commit par exercice réussi (`git add l1-doubler.js`, `git commit -m "Ajoute la solution de l1"`).

Réussi si : les deux commandes affichent `i fail 0`, et VS Code ne souligne rien en rouge.

> ℹ Aides
>
> - **Le cours** : Lundi après-midi, « Callback et map », et le pas à pas, étapes 2 à 4.
> - **L'IA** : CS50.ai (palier IA 1) : demandez une explication du message d'erreur, pas le code.
> - **MDN** : Array.prototype.map().

Palier 2 — Majuscules et prix TTC (40 min)

1. `l5-majuscules.js` : la première lettre en majuscule.
2. Une étape se trouve dans MDN : la méthode des majuscules.
3. `l8-prix-ttc.js` : les prix TTC, arrondis au centime.

4. **Réussi si** : leurs tests au vert.

Palier 2 : recommandé, semi-guidé. Durée : 40 min (20 min par exercice).

1. **`l5-majuscules`** : `capitalizeAll(words)` met la première lettre de chaque mot en majuscule.
   - Étape documentation : dans MDN, page *String*, trouvez la méthode qui met un texte en majuscules.
   - Pour un mot `word` : sa première lettre est `word[0]` (attention au mot vide !), le reste est `word.slice(1)`.
   - Écrivez une fonction `capitalize(word)` pour un seul mot, puis passez-la à `map`.
   - Lancez `node --test l5-majuscules.test.js`.
2. **`l8-prix-ttc`** : `pricesWithTax(prices)` renvoie chaque prix multiplié par 1,2, arrondi au centime.
   - L'arrondi est donné dans la consigne : `Math.round(price * 1.2 * 100) / 100` (MDN Web Docs, s. d.-e).
   - Lancez `node --test l8-prix-ttc.test.js`.

Réussi si : `node --test l5-majuscules.test.js` et `node --test l8-prix-ttc.test.js` affichent `ℹ fail 0`.

> Indice — le mot vide
>
> `''[0]` vaut `undefined` : `undefined.toUpperCase()` provoque une `TypeError`. Commencez `capitalize` par : si le mot est vide, renvoyez-le tel quel.

> ℹ Aides
>
> - **Le cours** : Lundi après-midi, « Callback et map » ; le pas à pas, étape 1 (`toUpperCase`).
> - **L'IA** : CS50.ai (palier IA 1) : demandez ce que fait `slice(1)` sur un texte, pas la solution.
> - **MDN** : String ; Array.prototype.map() ; Math.round().

Palier 3 — map à la main (30 min)

- Écrivez `myMap(array, callback)`, sans la méthode `map`.

- **Réussi si** : `node --test tp-my-map.defi.test.js` au vert.

Palier 3 : autonome. Durée : 30 min.

Dans `tp-my-map.js`, écrivez `myMap(array, callback)` : elle fait exactement ce que fait `array.map(callback)`, mais **sans** utiliser `map`. Vous connaissez déjà tout ce qu'il faut (semaine 3) ; à vous de trouver comment. Comprendre ce que fait `map` à l'intérieur, c'est ne plus jamais l'utiliser au hasard.

Réussi si : `node --test tp-my-map.defi.test.js` affiche `ℹ fail 0` (4 tests, dont « le tableau reçu n'est pas modifié »).

> ℹ Autonome
>
> Vous allez bloquer : c'est voulu, c'est le métier. Méthode : 1. lisez le message d'erreur en entier ; 2. cherchez-le dans la fiche « Lire un message d'erreur » ; 3. lisez la documentation ; 4. après 15 minutes bloqué sur le même point, demandez.

> ℹ Aides
>
> - **Le cours** : Lundi après-midi, « Callback et map » ; le visualiseur pas à pas, exemple `map`.
> - **L'IA** : CS50.ai (palier IA 1) : demandez comment parcourir un tableau et remplir un nouveau tableau, pas le code de `myMap`.
> - **MDN** : Array.prototype.map(), section *Description*.

Enregistrer et envoyer (2 min)
Même rituel, depuis `semaine-07` : `git status`, `git add` de vos fichiers, `git commit -m "…"` (par exemple « Ajoute la solution de l5 »), puis `git push`. **Réussi si** : `git status` affiche `rien à valider, la copie de travail est propre`.

À retenir

À retenir

- **Callback** : une fonction donnée à une autre, sans parenthèses.
- `map` : un nouveau tableau, de même longueur, un résultat par élément.
- **Le tableau reçu** ne change pas : `map` ne le modifie jamais.

Si le temps le permet

- Les paliers 2 et 3 non terminés.
- Le parcours d'autonomie, ligne 4.

Le visualiseur pas à pas : rejouez l'exemple `map` ligne par ligne, puis prédisez chaque résultat avant de cliquer sur **Ligne suivante**.

Débrief

- Pourquoi écrit-on `map(addTax)` et non `map(addTax())` ?
- Après un `map`, combien d'éléments dans le résultat ?

Sources

Etalab. s. d. « Licence Ouverte / Open Licence ». Consulté le 26 septembre 2026. https://www.etalab.gouv.fr/wp-content/uploads/2014/05/Licence_Ouverte.pdf.

MDN Web Docs. s. d.-a. « Array : méthode map() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/map.

MDN Web Docs. s. d.-b. « Fonction de rappel (callback) - Glossaire ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Glossary/Callback_function.

MDN Web Docs. s. d.-c. « Fonctions - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Functions.

MDN Web Docs. s. d.-d. « Fonctions fléchées - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Functions/Arrow_functions.

MDN Web Docs. s. d.-e. « Math : méthode statique round() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Math/round.

Ministère de la Culture. s. d. « Fréquentation des Musées de France (data.gouv.fr) ». Consulté le 26 septembre 2026. https://www.data.gouv.fr/datasets/frequentation-des-musees-de-france-1.